COMP OpenAI Access Guide

Version 3.11

Date: 13 January 2026

Revision History:

Revision	Date	Description
Version 3.11	13 Jan 2026	Updated pricing of models.
Version 3.10	12 Jan 2026	Updated supported models.
Version 3.9	9 Jan 2026	Updated supported models.
Version 3.8	11 Nov 2025	Updated supported models.
Version 3.7	18 Sep 2025	Updated supported models.
Version 3.6	8 Aug 2025	Added GPT-5-mini, GPT-5-nano, GPT-5-chat.
Version 3.5	28 Jul 2025	Removed GPT-4.5 (Preview).
Version 3.4	9 Jun 2025	Revised the model. GPT-4 is removed.
Version 3.3	22 Apr 2025	Revised the model version.
Version 3.2	10 Apr 2025	Added GPT 4.5 (Preview). Revised the model version.
Version 3.1	25 Nov 2024	Added GPT 4o mini. Revised the model version. And updated the default API version.
Version 3	12 Jun 2024	Added GPT 4o and revised the document.
Version 2.2	8 Apr 2024	Added example on accessing the API gateway in Python using requests module
Version 2.1	13 Mar 2024	Deployed gpt-3.5-turbo-instruct to the API gateway
Version 2	5 Mar 2024	Added GPT 4 and GPT 4 Turbo to the API gateway
Version 1	13 Jul 2023	Initial release

Table of Contents

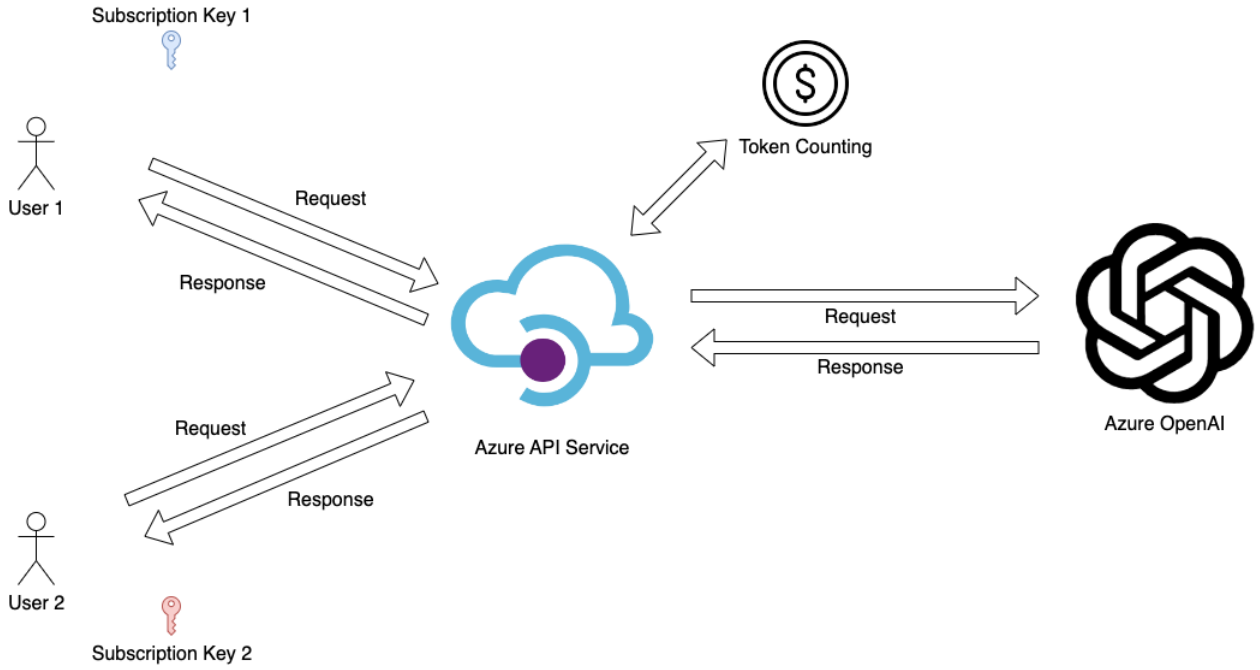
1. What is COMP OpenAI? 4
2. What is subscription key? Can I use it directly to access OpenAI API? 5
3. What API services does COMP OpenAI provide? 5
4. Can I fine-tune/ customize models using my own training data set? 5
5. What models are currently deployed for chat completions or legacy completions? 5
6. What is the information for accessing COMP OpenAI REST API? 6
7. What software do I need to access COMP OpenAI? 7
8. What is the limitation of the COMP OpenAI? 7
9. Why is “used tokens” different from the returned “total tokens” in an API call? 8
10. Legacy text completion usage and example (HTTP Request) 8
11. Chat Completion usage and examples (HTTP Request) 13
12. Embeddings API usage and example (HTTP Request) 18
13. How do I know my current used tokens and remaining tokens? 20
14. How do I use Python OpenAI library (version 1.x) to connect to OpenAI API gateway? 21
15. How do I include image in GPT-4o query? 23
16. Why I received status code 500 with message “Internal Server Error”? 24
17. Why I received status code 429? 24
18. How to estimate tokens? Are there any tools to estimate tokens? 24
19. Are there any GUI tools to evaluate the COMP OpenAI API gateway? 25
20. What is the difference between OpenAI API and Azure OpenAI API and COMP OpenAI API gateway? 26
21. Resources 27
22. References 28

1. What is COMP OpenAI?

COMP OpenAI is a set of RESTful API for staff to access OpenAI services and integrate to their applications. Recently, the Department has subscribed the Microsoft Azure OpenAI service. The Azure OpenAI service is a pay-as-you-go service and the fee is counted by something called token. Both the input text and output text will be counted towards tokens. The total tokens used will be returned for every OpenAI API request. You are advised to keep record of the total tokens in your applications. Below is the estimation of calculating tokens[1]:

1 token ~= 4 chars in English 1 token ~= ¾ words 100 tokens ~= 75 words	OR	1-2 sentence ~= 30 tokens 1 paragraph ~= 100 tokens 1,500 words ~= 2048 tokens

In order to count the total number of tokens used, an API frontend service is necessary and deployed. All applications are required to access OpenAI service through the API frontend service. The below image illustrates this situation.



Each user will be given a different subscription key. The user requires to use the key to call the API frontend service and the API frontend service will forward the request to underlying Azure OpenAI service. You may refer to the Azure OpenAI REST API reference[2] for latest available options and requirements of the API call. This guide will give you examples with curl to demonstrate how to call the API frontend service. But you can use any HTTPS client and library which support custom headers and body with JSON formatted parameters to call the API. At this moment, ChatGPT (GPT 3.5) for completions and chat completions,

[1] https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them
[2] https://learn.microsoft.com/en-us/azure/cognitive-services/openai/reference

GPT 4, GPT 4 Turbo, GPT 4o and GPT 4o Mini for chat completions and gpt-35-instruct for embeddings are deployed.

Please be reminded, like other RESTful API, fundamental knowledge on HTTP is necessary. You should have knowledge about what is HTTP request and response. This is important for you to construct the API call and process the returned result from the server in your program.

2. What is subscription key? Can I use it directly to access OpenAI API?

The subscription key provided is valid only in **COMP OpenAI API gateway**. The key must be provided in request header with header name "api-key" (without the quote). The key is **NOT** valid to access OpenAI API or other Microsoft Azure OpenAI services.

3. What API services does COMP OpenAI provide?

The below services are provided:

- Chat Completions API
- Legacy Completions API
- Embeddings

4. Can I fine-tune/ customize models using my own training data set?

No. Recently Microsoft turned off fine-tuning/ customizing models worldwide.[3]

5. What models are currently deployed for chat completions or legacy completions?

The below table listed out models which are currently deployed in COMP Azure OpenAI environment.

Model	Deployment Name	Version[%]	Reference Price (per 1000 tokens, in US$)*	Scale Factor (Based on ChatGPT price: $0.002 per 1000 tokens)*
GPT-4o[$]	gpt4o	2024-11-20	$0.011	5.5
GPT-4o-mini[$]	gpt4o-mini	2024-07-18	$0.00066	0.33
GPT-4.1[$] (Global)	gpt41-g	2025-04-14	$0.008	4
GPT-4.1-mini[$]	gpt41-mini	2025-04-14	$0.0016	0.8
GPT-4.1-nano[$]	gpt41-nano	2025-04-14	$0.0004	0.2
GPT-5[$] (Global)	gpt5-g	2025-08-07	$0.01	5

[3] https://learn.microsoft.com/en-us/azure/cognitive-services/openai/concepts/models#gpt-3-models

GPT-5-mini$ (Global)	gpt5-mini-g	2025-08-07	$0.002	1
GPT-5-nano$ (Global)	gpt5-nano-g	2025-08-07	$0.0004	0.2
GPT-5-chat$ (Global)	gpt5-chat-g	2025-08-07	$0.01	5
GPT-5.1$ (Global)	gpt51-g	2025-11-13	$0.01	5
GPT-5.2$ (Global)	gpt52-g	2025-12-11	$0.014	7
GPT-5.2-chat$ (Global)	gpt52-chat-g	2025-12-11	$0.014	7
o1$+ (Global)	o1-g	2024-12-17	$0.06	30
o3$+ (Global)	o3-g	2025-04-16	$0.008	4
o3-mini$+	o3-mini	2025-01-31	$0.00484	2.42
o4-mini$+	o4-mini	2025-04-16	$0.0044	2.2

* Reference price and scale factor may be adjusted from time to time. The total tokens returned from Azure OpenAI backend service will be multiplied by the scale factor and stored in COMP API gateway because of the price different. The calculated used tokens will be rounded to the smallest integer that is larger than the calculated value.

^ Can be used in both chat completions or legacy completions API

Can be used in legacy completions API only

$ Can be used in chat completions API only

% The version of models may update automatically by Microsoft

+ The thinking part when generating response (reasoning tokens) counts towards completion tokens.

6. What is the information for accessing COMP OpenAI REST API?

Please replace “{Base URL}” and “{Deployment Name}” to the proper value. The “{Deployment Name}” can be found in previous section.

Base URL	https://comp.azure-api.net/azure
Authentication	Include a header named “Api-key” with the subscription key provided as the value.
Services/ Endpoints	Legacy Completions API URL: {Base URL}/openai/deployments/{Deployment Name}/completions Method: POST Required Header: “Content-type: application/json”
	Chat Completions API URL: {Base URL}/openai/deployments/{Deployment Name}/chat/completions Method: POST Required Header: “Content-type: application/json”
	Embeddings API URL: {Base URL}/openai/deployments/{Deployment Name}/embeddings Method: POST

	Required Header: “Content-type: application/json”
	Query used tokens and remaining tokens URL: {Base URL}/openai/used-tokens Method: GET
API Version	By default, the API frontend set the OpenAI API version to 2024-10-21. If you want to call the API with different version, you can append the API version as query string with parameter name “api-version” to the API frontend URL like Azure OpenAI REST API. For the list of supported API version, please refer to Azure OpenAI Service REST API reference[2].

For example, to query the total used tokens, you can make a HTTPS request to https://comp.azure-api.net/azure/openai/used-tokens together with your subscription key. Here is an example of calling the API by curl (Replace “{My API Key}” with your subscription key provided.):

```
$ curl https://comp.azure-api.net/azure/openai/used-tokens -i -H "Api-key: {My API Key}"
```

with response:

```
HTTP/1.1 200 OK
Content-Length: 18
Content-Type: application/json
Date: Fri, 16 Jun 2023 02:42:12 GMT
Cache-Control: private

{"UsedTokens":372,"RemainingTokens":19628}
```

7. What software do I need to access COMP OpenAI?

To access COMP OpenAI, a HTTP client which supports HTTPS connection, custom request headers and body is needed. Most programming languages do have some HTTP clients available. The table below lists some of them.

Programming Language	HTTP Client Package
Python	requests module
PHP	Guzzle
Java	java.net.http.HttpClient (Java SE 11 or later) Apache HttpClient (Java SE 8)

8. What is the limitation of the COMP OpenAI?

As the frontend service are used to count for the total number of tokens used, COMP has set up below limitation for such purpose.

- Maximum 1 concurrent connection is allowed.

- The “stream” parameter[4] in the backend Azure OpenAI REST API will always be false.

Microsoft have also set up some limitations on Azure OpenAI. The maximum tokens per request for ChatGPT (GPT 3.5 Turbo) is limited to 4,096 tokens including both prompt tokens and completion tokens, 8192 tokens for GPT 4, 4096 output tokens for GPT 4 Turbo and GPT 4o. For Ada002 version 2 for embeddings, the maximum tokens per request is limited to 8,191 tokens. Microsoft has also set up the maximum tokens per minute per model per region (E.g. 240,000 tokens for GPT 3.5 Turbo, 20,000 or 40,000 for GPT 4, 80,000 for GPT 4 Turbo).

9. Why is “used tokens” different from the returned “total tokens” in an API call?

Because prices of different models are different and the difference is huge. The “total tokens” returned in the API call will be multiplied by a scaling factor to limit the total amount to the specific value to avoid budget overruns.

10. Legacy text completion usage and example (HTTP Request)

Completions is the legacy component of the OpenAI service. Using GPT-4 model in Completions API is not supported. Thus, you should avoid using this API in your application. Please use the Chat Completions API if possible. The model will generate text completions based on the input prompt. To access the completions service, you will need to initialize a HTTP request to the endpoint and construct required header and body and optional parameters. Below is the information to access the completions API. The request body must be in JSON.

Supported Model:
ChatGPT (GPT 3.5 Turbo)

HTTP Request:
POST https://comp.azure-api.net/azure/openai/deployments/{Deployment Name}/completions

* Replace “{Deployment Name}” to the deployment name of the model listed in the above table.

Optional Query String Parameters:		
Name	**Default**	**Description**
api-version	2023-05-15	The supported version of the Azure OpenAI REST API.

Required Headers:	
Name	**Description**

[4] https://learn.microsoft.com/en-us/azure/cognitive-services/openai/reference

Api-key	Provide your subscription key here.
Content-type	Must be “application/json”.

Request Body (in JSON format):				
Name	**Type**	**Required?**	**Default**	**Description**
prompt	String or array	Optional	<|endoftext|>	The input prompt text submitted to OpenAI for generating completions text.
max_tokens	Integer	Optional	16	The maximum number of tokens to generate by model per completion. The maximum tokens are limited by the models which include both prompt tokens and completions tokens. That is the maximum tokens for ChatGPT is 4,096.
temperature	Number	Optional	1	Value is between 0 and 2. Higher values will generate more creative content and 0 for ones with a well-defined answer.
top_p	Number	Optional	1	Alternative sampling with temperature called nucleus sampling. The model will consider result of the top_p probability mass. i.e. for value 0.1, the model will only consider the top 10% probability mass. According to the official documentation, users are not recommended altering both temperature and top_p paramaters.
logit_bias	Map	Optional	null	Modify the likelihood of specified tokens appearing in the completion. Accepts a json object that maps tokens (specified by their token ID in the GPT tokenizer) to an associated bias value from -100 to 100. You can use this tokenizer tool (which works for both GPT-2 and GPT-3) to convert text to token IDs. Mathematically, the bias is added to the logits generated

				by the model prior to sampling. The exact effect will vary per model, but values between -1 and 1 should decrease or increase likelihood of selection; values like -100 or 100 should result in a ban or exclusive selection of the relevant token. As an example, you can pass {"50256": -100} to prevent the <|endoftext|> token from being generated.
n	Integer	Optional	1	Number of completions to be generated for each prompt. Please be reminded about the maximum tokens per request of model.
suffix	String	Optional	null	The suffix that comes after a completion of inserted text.
stop	String or array	Optional	null	Can set up to four sequences to stop the model to generate further tokens. You may refer to "How do I use Stop Sequences" for examples.
presence_penalty	Number	Optional	0	Number between -2.0 and 2.0. Positive values penalize new tokens based on whether they appear in the text so far, increasing the model's likelihood to talk about new topics.
frequency_penalty	Number	Optional	0	Number between -2.0 and 2.0. Positive values penalize new tokens based on their existing frequency in the text so far, decreasing the model's likelihood to repeat the same line verbatim.

* Please refer to the Azure OpenAI Service REST API reference for more information.

curl Example (Replace "{YOUR_API_KEY}" with your subscription key provided.):

```
$ curl https://comp.azure-api.net/azure/openai/deployments/gpt35/completions -H "Content-Type: application/json" -H "api-key: {YOUR_API_KEY}" -d "{
  \"prompt\": \"Once upon a time\",
  \"max_tokens\": 5,
```

```
  \"temperature\": 1.5,
  \"n\": 2
}"
```

* To run the above command in Windows, please run it inside WSL of Windows. Multiline is not supported in Windows command prompt.

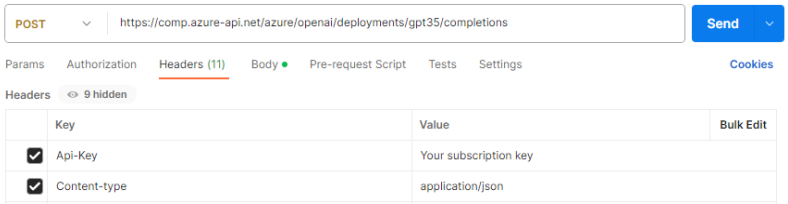
The response will be a JSON text like on successful: (Please note that the actual output is a single block of text without line breaks.)


```
{
    "id":"cmpl-7T5UhWAVg7qEeSSkTNJA5pDfdIklK",
    "object":"text_completion",
    "created":1687166987,
    "model":"gpt-35-turbo",
    "choices":[
        {
            "text":" psychologists conducted research on motivation",
            "index":0,
            "finish_reason":"length",
            "logprobs":null
        },
        {
            "text":", Jack conquered big enemies",
            "index":1,
            "finish_reason":"length",
            "logprobs":null
        }
    ],
    "usage":{
        "completion_tokens":10,
        "prompt_tokens":4,
        "total_tokens":14
    }
}
```

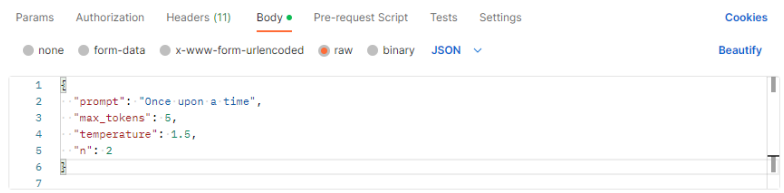

Please note about the completion_tokens. If you set max_tokens to 5, this value will be applied to each completion. You should record the value of total_tokens as this is what will be billed.

Postman on Windows example:

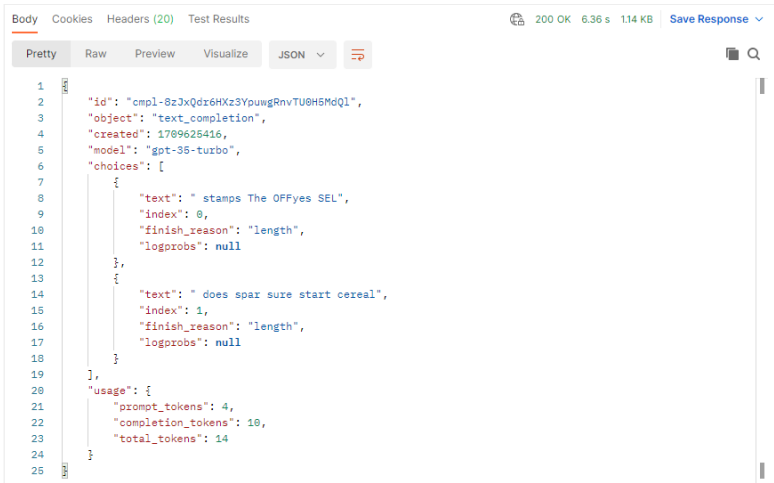
1. Set the URL and the required headers.



2. Set the request body:



3. Click “Send” button and wait the response.
4. The response looks like below on successful.



5. Please note about the completion_tokens. If you set max_tokens to 5, this value will be applied to each completion. You should record the value of total_tokens as this is what will be billed.

Python request module example (Replace “{YOUR_API_KEY}” with your subscription key provided.):

```
import requests
base = "https://comp.azure-api.net/azure/"
apiKey = "{YOUR_API_KEY}"
deployName = "gpt35"
body = {"prompt": "my text here"}]}

response = requests.post(base + "/openai/deployments/" + deployName + "/chat/completions",
json=body, headers={"api-key": apiKey})

jsonObj = response.json()

# only valid on successful
```

```
print(jsonObj["choices"][0]["text"])
```

11. Chat Completion usage and examples (HTTP Request)

Chat completions is the replacement of the legacy completions service. The different is that a conversation is submitted to generate completion instead of a prompt. Please be reminded each API call is a standalone operation. You need to include previous conversation in an API call to generate the completion.

Supported Model:
ChatGPT (GPT 3.5 Turbo), GPT 4, GPT 4 Turbo, GPT 4o

HTTP Request:
POST https://comp.azure-api.net/azure/openai/deployments/{Deployment Name}/chat/completions

* Replace “{Deployment Name}” with the deployment name of the model listed in the table above.

Required Headers:	
Name	**Description**
Api-key	Provide your subscription key here.
Content-type	Must be “application/json”.

Request Body (in JSON format):				
Name	**Type**	**Required?**	**Default**	**Description**
messages	Array	Required		The messages to generate chat completion for. Each item in the array is a JSON object which include the below parameters:

Name	**Required?**	**Description**
role	Required	The role of the message author. One of "system", "user", "assistant"
content	Required	The content of the message. It can be a string or an array of content structure (for vision

				<table><tr><td></td><td></td><td>enabled model).</td></tr><tr><td>name</td><td>Optional</td><td>The name of the author of the message.</td></tr></table>
max_tokens	Integer	Optional	Max allowed tokens	The maximum number of tokens to generate by model. The maximum tokens are limited by the models which include both input tokens and completions tokens. That is the maximum tokens for ChatGPT is 4,096. * The max_tokens parameter does not apply to o1/o3/o4 series. Please use max_completion_tokens instead.
temperature	Number	Optional	1	Value is between 0 and 2. Higher values will generate more random and lower values will make it more focused and deterministic.
logit_bias	Object	Optional	null	Modify the likelihood of specified tokens appearing in the completion. Accepts a json object that maps tokens (specified by their token ID in the tokenizer) to an associated bias value from -100 to 100. Mathematically, the bias is added to the logits generated by the model prior to sampling. The exact effect will vary per model, but values between -1 and 1 should decrease or increase likelihood of selection; values like -100 or 100 should result in a ban or exclusive selection of the relevant token.
n	Integer	Optional	1	Number of completions to be generated. Please be reminded about the maximum tokens per request per model.
stop	String or array	Optional	null	Can set up to four sequences to stop the model to generate further tokens. You may refer to "How do I use Stop Sequences" for examples.
presence_penalty	Number	Optional	0	Number between -2.0 and 2.0. Positive values penalize new tokens based on whether they appear in the text so far, increasing the model's likelihood to talk about new topics.

frequency_penalty	Number	Optional	0	Number between -2.0 and 2.0. Positive values penalize new tokens based on their existing frequency in the text so far, decreasing the model's likelihood to repeat the same line verbatim.

* Please refer to the Azure OpenAI Service REST API reference for more information.

The content structure of the messages parameter.				
Name	**Type**	**Required?**	**Default**	**Description**
type	String	Required		The value is either "text" for text input or "image_url" for image input.
text	String	Required for text message		The text content of the message.
image_url	Object	Required for image		The structure with the following parameters:

Name	**Required?**	**Description**
url	Required	The URL of the image. It can be data URL[5].
detail	Optional	The value can be "high", "low", or "auto".

curl Example (Replace "{YOUR_API_KEY}" with your subscription key provided.):

```
$ curl https://comp.azure-api.net/azure/openai/deployments/gpt35/chat/completions -H "Content-Type: application/json" -H "api-key: {YOUR_API_KEY}" -d '{
  "messages": [
    {
      "role": "system",
      "content": "What can I help you?"
    },
    {
      "role": "user",
      "content": "Any food recommended for today breakfast?",
      "name": "Anita"
    },
    {
      "role": "assistant",
```

[5] Data URLs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/Data_URLs

```
        "content": "Sure! Here are a few breakfast food recommendations for today: \n\n1. Overnight oats topped with berries, bananas, nuts, and almond butter. \n"
      },
      {
        "role": "user",
        "content": "How about today dinner?",
        "name": "Anita"
      }
  ],
  "max_tokens": 100,
  "stop": "2.",
  "temperature": 1.5
}'
```

* To run the above command in Windows, please run it inside WSL of Windows. Multiline is not supported in Windows command prompt.

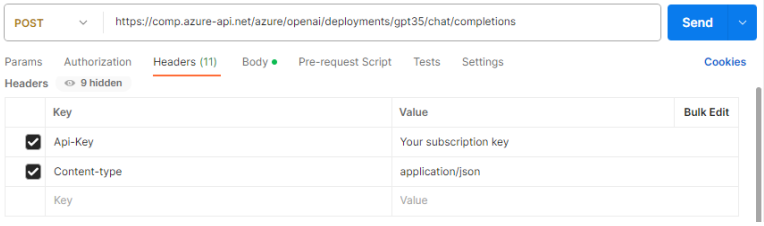
The response will be a JSON text like on successful: (Please note that the actual output is a single block of text without line breaks.)


```
{
  "id":"chatcmpl-7TL13QD0ofybim2HYZhqt0dEd6f9K",
  "object":"chat.completion",
  "created":1687226653,
  "model":"gpt-35-turbo",
  "choices":[
    {
      "index":0,
      "finish_reason":"stop",
      "message":{
        "role":"assistant",
        "content":"For dinner, you can try one of the following options:\n\n1. Grilled chicken with vegetables, such as asparagus and Baker potatoes.\n\n"
      }
    }
  ],
  "usage":{
      "completion_tokens":28,
      "prompt_tokens":73,
      "total_tokens":101
  }
}
```

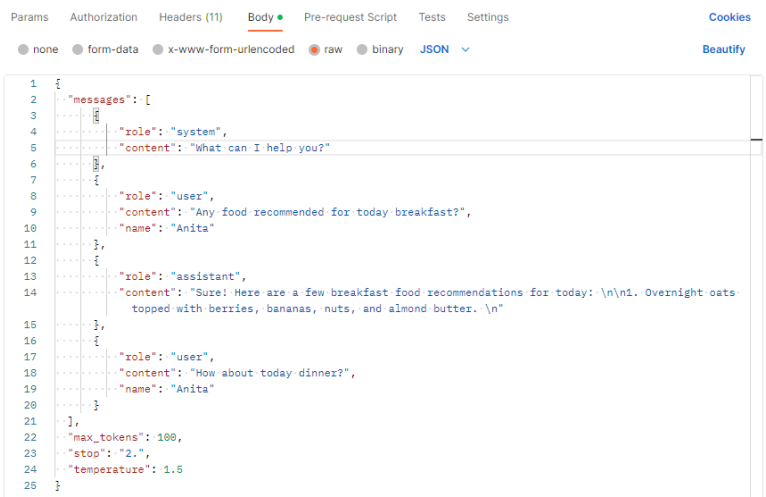

You should record the value of total_tokens as this is what will be billed.

Postman on Windows Example:

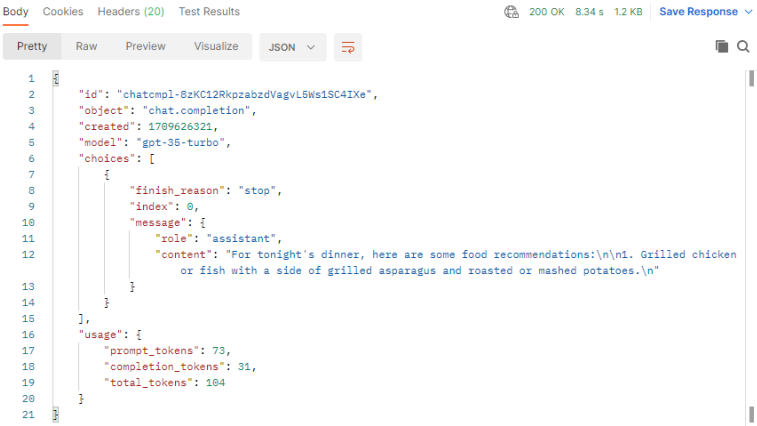
1. Set the URL and the required headers.



2. Set the request body:



3. Click “Send” button and wait the response.
4. The response looks like below on successful.



5. You should record the value of total_tokens as this is what will be billed.

Python request module Example (Replace “{YOUR_API_KEY}” with your subscription key provided.):

```
import requests
base = "https://comp.azure-api.net/azure/"
apiKey = "{YOUR_API_KEY}"
deployName = "gpt35"
body = {"messages":[{"role":"system","content":"What can I help
you?"},{"role":"user","content":"Any food recommended for today
```

```
breakfast?","name":"Anita"},{"role":"assistant","content":"Sure! Here are a few breakfast food recommendations for today: \n\n1. Overnight oats topped with berries, bananas, nuts, and almond butter. \n"},{"role":"user","content":"How about today dinner?","name":"Anita"}],"max_tokens":100,"stop":"2.","temperature":1.5}

response = requests.post(base + "/openai/deployments/" + deployName + "/chat/completions", json=body, headers={"api-key": apiKey})

jsonObj = response.json()

# only valid on successful
print(jsonObj["choices"][0]["message"]["content"])
```

12. Embeddings API usage and example (HTTP Request)

The embeddings service is to get a vector representation of some text that can be easily consumed by other machine learning models and algorithms. Please note that the output vector will not count for tokens.

Supported Model:
GPT-3.5-turbo-instruct (Model Name: gpt-35-turbo-instruct; Deployment Name: gpt35instruct)

HTTP Request:
POST https://comp.azure-api.net/azure/openai/deployments/{Deployment Name}/embeddings

* Replace “{Deployment Name}” with the deployment name of the model listed in the table above.

Required Headers:	
Name	**Description**
Api-key	Provide your subscription key here.
Content-type	Must be “application/json”.

Request Body (in JSON format):				
Name	**Type**	**Required?**	**Default**	**Description**
input	String	Required		The text to generate vector representation. Please take care of the maximum tokens of the models. The Azure OpenAI Service REST API reference suggested replacing newlines (\n) with a single space unless you are embedding code.

* Please refer to the Azure OpenAI Service REST API reference for more information.

cURL example (Replace “{YOUR_API_KEY}” with your subscription key provided.):

```
$ curl https://comp.azure-api.net/azure/openai/deployments/ada002/embeddings  -H "Content-Type: application/json"  -H "api-key: {YOUR_API_KEY}"  -d '{
  "input": "The text to generate vector representation. "
}'
```

* To run the above command in Windows, please run it inside WSL of Windows. Multiline is not supported in Windows command prompt.

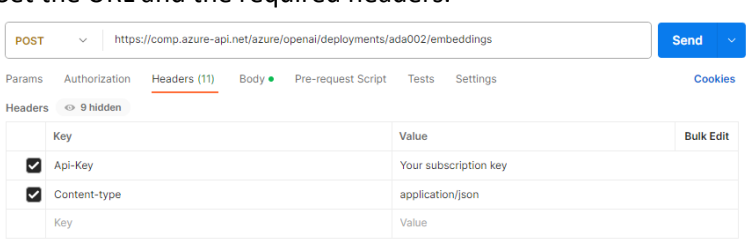
The response will be a JSON text like on successful:


```
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "index": 0,
      "embedding": [
        -0.029177554,
        0.0014902321,
        ....
        -0.0034778088,
        -0.04353568
      ]
    }
  ],
  "model": "ada",
  "usage": {
    "prompt_tokens": 7,
    "total_tokens": 7
  }
}
```


You should record the value of total_tokens as this is what will be billed.

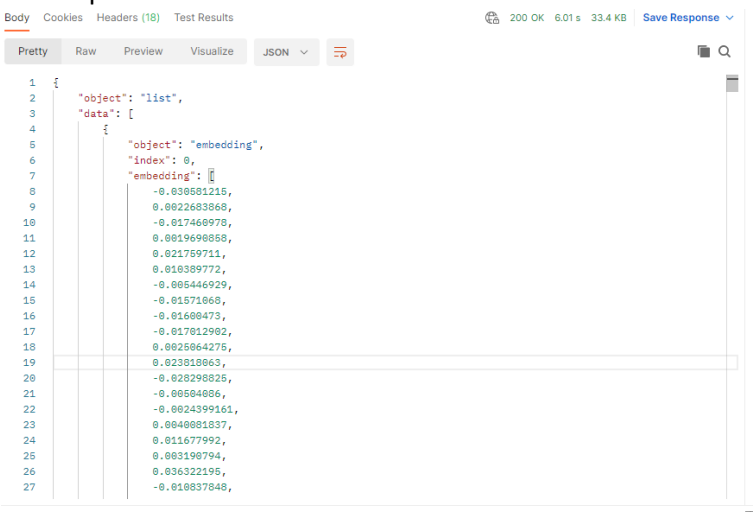
Postman on Windows example:

1. Set the URL and the required headers.



2. Set the request body:

3. Click “Send” button and wait the response.
4. The response looks like below on successful.



5. You should record the value of total_tokens as this is what will be billed.

13. How do I know my current used tokens and remaining tokens?

TT has created the API operation for you to query your total used tokens and remaining tokens. Below is the information. The used tokens were scaled by the scaling factor of the specific model list above.

HTTP Request:
GET https://comp.azure-api.net/azure/openai/used-tokens

Required Headers:	
Name	**Description**
Api-key	Provide your subscription key here.

Using curl as an example (Replace “{YOUR_API_KEY}” with your subscription key provided.):

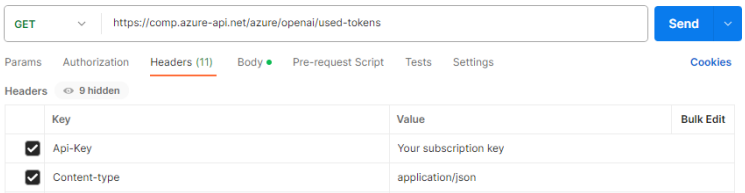
```
$ curl https://comp.azure-api.net/azure/openai/used-tokens -H "api-key: {YOUR_API_KEY}"
```

The response will be a JSON text like on successful:

```
{"UsedTokens":0,"RemainingTokens":0}
```

Using Postman on Windows as an example:

1. Set the URL and the required headers.



2. Click "Send" button and wait the response.
3. The response looks like below on successful.

Using requests module in Python as an example (Replace "{YOUR_API_KEY}" with your subscription key provided.):

```
import requests
base = "https://comp.azure-api.net/azure/"
apiKey = "{YOUR_API_KEY}"
deployName = "ada002"
body = {"input": "The text to generate vector representation."}]}

response = requests.post(base + "/openai/deployments/" + deployName + "/embedding", json=body,
headers={"api-key": apiKey})

jsonObj = response.json()
print(jsonObj)
```

14. How do I use Python OpenAI library (version 1.x) to connect to OpenAI API gateway?

1. Install Python OpenAI API package using pip command if Python OpenAI API package has not installed yet: (If pip command is not found, please refer to this: https://pip.pypa.io/en/stable/installation/ to install it)

```
pip install openai
```

2. Import the package and set up the OpenAI API gateway resource endpoint and key in Python script: (Change "{YOUR_API_KEY}" to your subscription key provided. In this example, we use "gpt4o" as the model. Replacing it with the deployment name of the model which you want to use. The deployment name of the model can be found at previous session.)

```
from openai import AzureOpenAI

base = "https://comp.azure-api.net/azure"
```

```
apiKey = "{YOUR_API_KEY}"
deployName = "gpt4o"
apiVersion = "2024-02-01"

client = AzureOpenAI(
    api_key=apiKey,
    api_version=apiVersion,
    base_url=f"{base}/openai/deployments/{deployName}",
)
```

3. Submit the query with content inside the "messages" parameter. The "messages" parameter is an array of messages between system and user. (The below example uses the Chat Completions API.)

```
response = client.chat.completions.create(
    model=deployName,
    messages=[
        # message 1
        { "role": "system", "content": "What can I help you?" },
        # message 2
        { "role": "user", "content": "What is the place must be visit when I travelling to
Hong Kong?" }
    ]
)
```

4. At last, get the generated text:

```
print(response.choices[0].message.content)
```

5. The possible output after running the script is:

```
Visiting Hong Kong offers a multitude of experiences, blending tradition and modernity.
Here are some must-visit places:

1. **Victoria Peak**: Also known as The Peak, it offers stunning panoramic views of Hong
Kong’s skyline, Victoria Harbour, and the surrounding islands. You can take the historic
Peak Tram to get there.

2. **Victoria Harbour**: Famous for its nightly light show, "A Symphony of Lights," and the
Star Ferry, which offers a scenic boat ride between Kowloon and Hong Kong Island.

3. **Tsim Sha Tsui Promenade**: Offers excellent views of the skyline, especially at night.
Don't miss the Avenue of Stars, which celebrates Hong Kong's film industry.

4. **Big Buddha (Tian Tan Buddha)**: Located on Lantau Island near Ngong Ping Village,
this enormous statue is a major attraction. You can reach it via a scenic cable car ride.

5. **Ngong Ping 360**: The cable car ride to Ngong Ping Village on Lantau Island provides
expansive views of the South China Sea and lush mountains.
```

6. **Hong Kong Disneyland**: A magical destination that's fun for families and Disney fans alike. It's smaller than its American counterparts but still packed with attractions.

7. **Ocean Park**: A marine mammal park, oceanarium, animal theme park, and amusement park, located in Wong Chuk Hang and Nam Long Shan.

8. **Temples**: Man Mo Temple on Hollywood Road, Wong Tai Sin Temple in Kowloon, and Chi Lin Nunnery in Diamond Hill are significant religious sites worth visiting.

9. **Lan Kwai Fong and SoHo**: These areas are famous for their bustling nightlife, dining, and entertainment options.

10. **Central District**: This area is the business heart of Hong Kong, characterized by its mix of modern skyscrapers and colonial buildings. Don't miss the Central-Mid-Levels Escalator, the world's longest outdoor covered escalator system.

11. **Stanley Market and Murray House**: Stanley is a coastal village known for its market which is a great place for bargain shopping.

12. **Repulse Bay**: A popular beach destination that combines beautiful scenery with a relaxing atmosphere.

13. **Sai Kung**: Known as the "Back Garden of Hong Kong," it's great for outdoor activities like hiking, seafood dining, and visiting the beautiful islands.

14. **Kowloon Walled City Park**: Once the site of a densely populated and largely ungoverned settlement, it's now a peaceful and historical park.

Each of these places offers a unique slice of what makes Hong Kong such a vibrant and dynamic city. Whether you're interested in history, nature, or modern attractions, you'll find something that fascinates you.

15. How do I include image in GPT-4o query?

GPT-4o now accepts both text and image as the input for generating text-based output. The below is the example of the JSON object to be submit to the API that includes image in the query.

```
{
 "messages": [
    {
       "role": "system",
       "content": "What can I help you?"
    },
    {
       "role": "user",
       "content": [
```

```
            {
              "type": "text", "text": "What is the name of the organisation of the image?"
            },
            {
               "type": "image_url", "image_url": {"url": "https://www.polyu.edu.hk/assets/img/main-logo-
1x.png"}
            }
        ],
        "name": "Anita"
    }
  ],
  "max_tokens": 100
}
```

Please have a look on the second object in “messages” array. Prepare an object with “type” set to “image_url” and another object with the URL of the image set to “image_url”. The URL of the image can be a HTTP or HTTPS URL or a data URL.

16. Why I received status code 500 with message “Internal Server Error”?

There are many reasons that the COMP OpenAI returns status code 500 with message “Internal Server Error”. In most cases, either you called an invalid endpoint (i.e. wrong URL), the “api-key” header is missing or your subscription key is invalid. Please verify all of them are correct.

Example of the response with status code 500:

```
{ "statusCode": 500, "message": "Internal server error", "activityId": …. }
```

17. Why I received status code 429?

There are many reasons that the COMP OpenAI returns status code 429. Please refer to the message for the reason. The below table lists out some possible messages you may get.

Message of status code 429	**Description**
Concurrency limit exceeded.	COMP OpenAI does not allow concurrent connection. It is due to recording used tokens for billing purposes.
All tokens used. Total Used tokens: 0	All tokens have been used up. Further request will not be allowed.

18. How to estimate tokens? Are there any tools to estimate tokens?

The tokens can be roughly estimated by the below information:

1 token ~= 4 chars in English 1 token ~= ¾ words 100 tokens ~= 75 words	OR	1-2 sentence ~= 30 tokens 1 paragraph ~= 100 tokens 1,500 words ~= 2048 tokens

There are some tools you can use to estimate tokens.

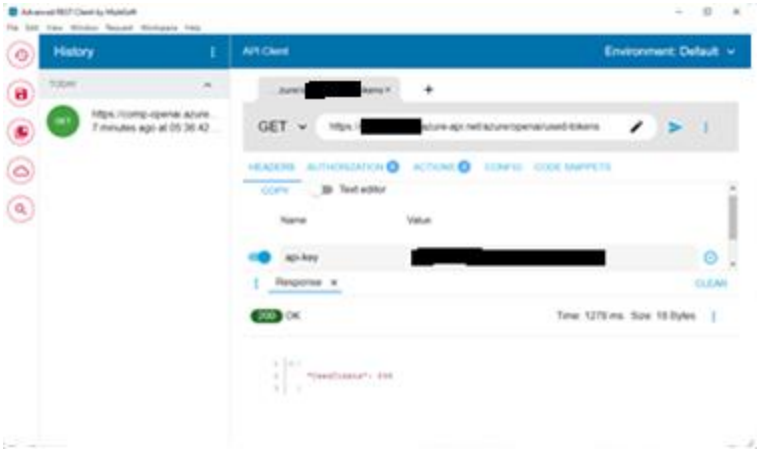
1. Web based UI created by OpenAI called Tokenizers (https://beta.openai.com/tokenizer)
2. Python module called Tiktoken (https://github.com/openai/tiktoken)
3. Python class called GPT2TokenizersFast (https://huggingface.co/docs/transformers/model_doc/gpt2#transformers.GPT2TokenizerFast)
4. Node.js module called gpt-3-encoder (https://www.npmjs.com/package/gpt-3-encoder)
5. PHP library called GPT-3-Encoder-Decoder-PHP (https://github.com/CodeRevolutionPlugins/GPT-3-Encoder-PHP)

For the usage of the tools, please refer to the related documentation.

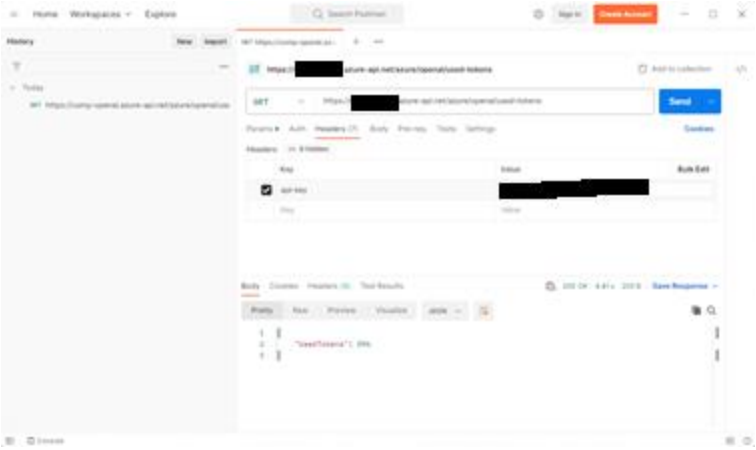
19. Are there any GUI tools to evaluate the COMP OpenAI API gateway?

There are tons of REST API clients available for you to evaluate the API. Here are some examples:

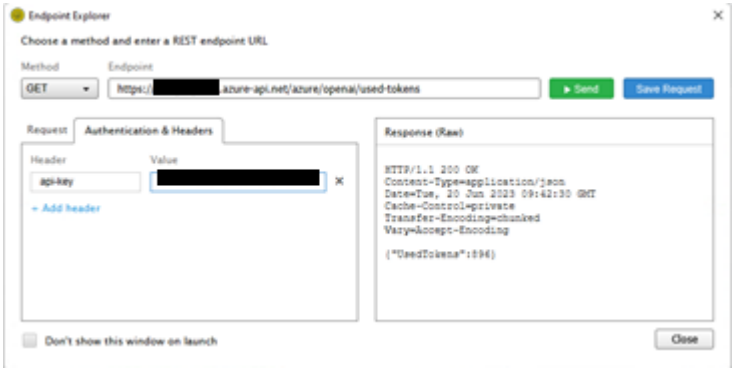
1. Desktop client (Support Windows, MacOS and Linux): Advanced REST Client (https://install.advancedrestclient.com/home)



2. Desktop client (Support Windows, MacOS, Linux)/Web UI: Postman (https://www.postman.com/downloads/)



3. Desktop client (Support Windows, MacOS, Linux): SoapUI (https://www.soapui.org/tools/soapui/)



20. What is the difference between OpenAI API and Azure OpenAI API and COMP OpenAI API gateway?

The request body of these API are almost compatible, except that OpenAI API requires the presence of "model" parameter in the JSON of the request body. Other differences are the endpoint URL and authentication method. The below summarizes the difference.

1. Authentication:

OpenAI API	Header "Authorization" with value "Bearer OPENAI_API_KEY"
Azure OpenAI	Header "Api-key" with value "AZURE_API_KEY"
COMP OpenAI API	Header "Api-key" with value "Your subscription key"

2. Completions (legacy) endpoint:

OpenAI API	https://api.openai.com/v1/completions
Azure OpenAI	https://{resource-name}.openai.azure.com/openai/deployments/{deployment-id}/completions?api-version={api-version}

COMP OpenAI API	https://comp.azure-api.net/azure/openai/deployments/{Deployment Name}/completions

3. Chat completions endpoint:

OpenAI API	https://api.openai.com/v1/chat/completions
Azure OpenAI	https://{your-resource-name}.openai.azure.com/openai/deployments/{deployment-id}/chat/completions?api-version={api-version}
COMP OpenAI API	https://comp.azure-api.net/azure/openai/deployments/{Deployment Name}/chat/completions

4. Embeddings endpoint:

OpenAI API	https://api.openai.com/v1/embeddings
Azure OpenAI	https://{your-resource-name}.openai.azure.com/openai/deployments/{deployment-id}/embeddings?api-version={api-version}
COMP OpenAI API	https://comp.azure-api.net/azure/openai/deployments/{Deployment Name}/embeddings

* Texts with blanket {} are parameters that proper values must be substituted into them.

If your application supports OpenAI or Azure OpenAI and has options to change the endpoint and also the authentication method, change application settings accordingly to COMP OpenAI API gateway and your application should work.

21. Resources

- Most REST APIs are based on HTTP to communicate between client and server. It is recommended to understand the basic information of HTTP. For an overview on HTTP, this page: https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview is a good start.
- Information about Python requests module can be found at https://requests.readthedocs.io/en/latest/
- Information about PHP Guzzle library can be found at https://docs.guzzlephp.org/en/stable/
- Information about Java built-in HTTP Client (for Java SE 11 or later) can be found at https://docs.oracle.com/en%2Fjava%2Fjavase%2F11%2Fdocs%2Fapi%2F%2F/java.net.http/java/net/http/HttpClient.html
- Information about Apache HttpClient can be found at https://hc.apache.org/index.html
- Some applications to access and evaluate COMP OpenAI are:
    - curl (https://curl.se/)
    - Advanced REST Client (https://install.advancedrestclient.com/home)
    - Postman (https://www.postman.com/downloads/)
    - SoapUI (https://www.soapui.org/tools/soapui/)
- Information about data URL can be found at https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/Data_URLs

22. References

A. What is Azure OpenAI Service? - Azure Cognitive Services | Microsoft Learn (https://learn.microsoft.com/en-us/azure/cognitive-services/openai/overview)

B. Azure OpenAI Service REST API reference - Azure OpenAI | Microsoft Learn (https://learn.microsoft.com/en-us/azure/cognitive-services/openai/reference)

C. How do I use Stop Sequences? | OpenAI Help Center (https://help.openai.com/en/articles/5072263-how-do-i-use-stop-sequences)

D. OpenAI API Reference (https://platform.openai.com/docs/api-reference)